

PROGRAMMAZIONE II

TIPI E OGGETTI

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.A. 2025/2026

TIPIZZAZIONE DEI LINGUAGGI



TIP

Definizione informale di **tipo**: un insieme di valori insieme alle operazioni su di essi

Il tipo **integer** rappresenta

- ▶ Valori come -2 , -1 , 0 , 1 e 2
- ▶ Operazioni come $+$, $-$, $*$ e $/$

☹ Gli informatici teorici e i logici potrebbero non essere d'accordo

TIPI (CONT.)

Nella programmazione, i tipi definiscono un contratto tra variabili ed espressioni

Se una variabile ha tipo `integer`

- ▶ Deve/può contenere valori interi
- ▶ Deve/può essere confrontata con espressioni che producono valori interi
- ▶ Deve/può essere usata in contesti in cui sono attesi valori interi

La scelta tra deve e può spetta al linguaggio, sfortunatamente

LINGUAGGI DEBOLMENTE E FORTEMENTE TIPIZZATI

Python è un linguaggio **debolmente tipizzato**

- ▶ Una variabile non necessita di una dichiarazione di tipo
- ▶ Una variabile può contenere valori appartenenti a tipi differenti

```
x = 100;  
x = "Hello_World";  
x = (1,2,3,4);
```

// non importa

Java è un linguaggio **fortemente tipizzato**

- ▶ Una variabile richiede una dichiarazione di tipo
- ▶ Una variabile deve contenere solo valori appartenenti al tipo dichiarato

```
int x;  
x = 100;  
x = "Hello_World";
```

// errore di sintassi

LINGUAGGI DEBOLMENTE E FORTEMENTE TIPIZZATI (CONT.)

Tipizzazione debole

- ▶ Programmi più brevi, in generale
- ▶ Il programmatore ha più libertà, il linguaggio è più permissivo nell'applicare operazioni ai valori

Tipizzazione forte

- ▶ Il programmatore deve essere più disciplinato
- ▶ Più errori rilevati a tempo di compilazione

LINGUAGGI DEBOLMENTE E FORTEMENTE TIPIZZATI (CONT.)



TIPI PRIMITIVI

Il tipo `int` di Java

- I valori sono $-2^{31}, \dots, 2^{31} - 1$ e le operazioni sono $+$, $-$, $*$, $/$, $\%$, $-$

Qui, $\%$ è l'operazione modulo: $b \% c$ è il resto della divisione di b per c

- Se b vale 67 e c vale 60, allora $b \% c$ è 7

Qui, $-$ è **sovraccaricato (overloaded)**

- Può essere l'operazione di sottrazione: $b - c$
- Può essere l'operazione di negazione: $-b$

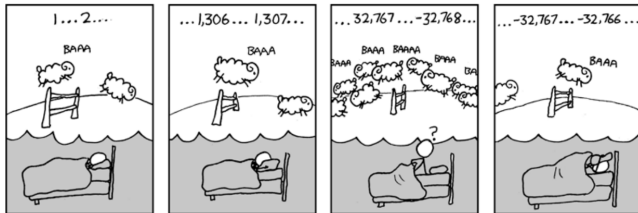
TIPI PRIMITIVI (CONT.)

Il tipo `int` di Java

- I valori sono $-2^{31}, \dots, 2^{31} - 1$ e le operazioni sono $+$, $-$, $*$, $/$, $\%$, $-$

Java: le operazioni di tipo `int` devono produrre un valore di tipo `int`

- Se a vale $2^{31} - 1$, allora $a + 1$ è -2^{31}



TIPI PRIMITIVI (CONT.)

Il tipo `int` di Java

- ▶ I valori sono $-2^{31}, \dots, 2^{31} - 1$ e le operazioni sono $+$, $-$, $*$, $/$, $\%$, $-$

Il tipo `double` di Java

- ▶ I valori sono come 2.51×10^{-6} o 24.9 , e le operazioni sono $+$, $-$, $*$, $/$, $\%$, $-$

Il tipo `char` di Java

- ▶ I valori sono come `'V'`, `'?'`, o `'\n'`, e nessuna operazione

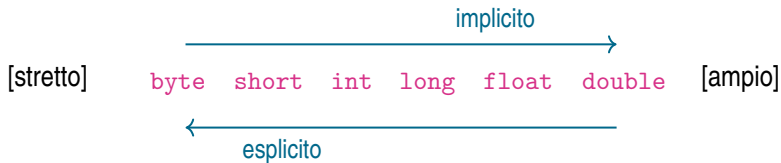
Il tipo `boolean` di Java

- ▶ I valori sono `true` o `false`, e le operazioni sono `!`, `&&`, `||` *// not, and, or*

CONVERSIONE DI TIPO

Cast **implicito**: `float f = 3;` converte l' `int 3` nel `float 3.0`

Cast **esplicito**: `int i = ( int ) 3.2;` converte il `double 3.2` nell' `int 3`



I CARATTERI SONO NUMERI

In realtà, il tipo `char` contiene interi a singolo byte

- ▶ `(int) 'V'` : rappresentazione Unicode in decimale, ossia 86 *// cast non necessario*
- ▶ `(char) 86` : rappresentazione decimale in Unicode, ossia 'V'

I caratteri codificano Unicode come valori letterali esadecimali a 16 bit

- ▶ `'\u0041'` è 'A', `'\u0056'` è 'V'
- ▶ `'\u0034'` è '\$', `'\u00A9'` è '©'

TIPI NON PRIMITIVI

In Java, tutto il resto

- ▶ Tutto è un oggetto
- ▶ Tranne i tipi primitivi

// Java è un linguaggio OO quasi puro

Il tipo `String` di Java

- ▶ I valori sono come “hello” o “world”, e le operazioni sono definite dall'utente
- ▶ Java rende disponibile una speciale operazione di **concatenazione** + su oggetti stringa
- ▶ Se a vale “Hello ” e b vale “World”, allora a + b è “Hello World”

OGGETTI JAVA



MEMORIA IN JAVA

Un oggetto rappresenta un'area di memoria isolata (sandboxed)

- ▶ Tale area è accessibile solo tramite passaggio di messaggi
- ▶ I campi sono sottoinsiemi di locazioni di memoria
- ▶ I metodi sono il modo in cui possiamo interagire con tali locazioni di memoria

In Java

- ▶ Tutto è un oggetto, quindi non ci sono variabili globali (a differenza del C)
- ▶ I programmatori non possono allocare/deallocare memoria manualmente (a differenza del C)
- ▶ La memoria di processo viene ottimizzata automaticamente dal **Garbage Collector** di Java

MEMORIA IN JAVA (CONT.)



PUNTATORI IN JAVA

Java ha i puntatori, ma non c'è aritmetica dei puntatori

- ▶ La gestione della memoria è molto vincolata, per mitigare gli errori del programmatore

In realtà Java usa i puntatori continuamente

- ▶ Per fare riferimento agli oggetti
- ▶ Per passare gli argomenti dei metodi (per valore)

Tecnicamente parlando, ogni variabile non primitiva in Java ha tipo `reference`

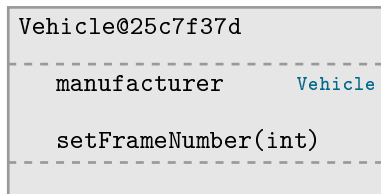
- ▶ Una variabile di tipo `String` conterrà un puntatore a un oggetto `String`

OGGETTI IN JAVA

Un **riferimento a un oggetto** contiene un nome di classe e un indirizzo di memoria

```
// myCar contiene Vehicle@null
```

```
// myCar contiene Vehicle@25c7f37d
```



I messaggi a un oggetto vengono inviati usando il suo riferimento (con dot-notation)

► myCar.setFrameNumber(2)

► Il messaggio viene inviato all'area di memoria 25c7f37d

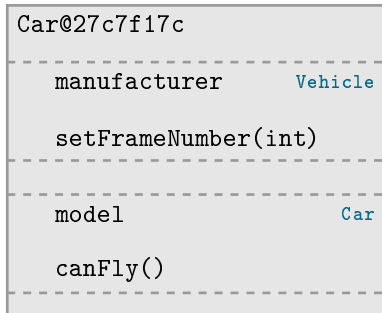
OGGETTI IN JAVA (CONT.)

Un riferimento a un oggetto contiene un nome di classe e un indirizzo di memoria

```
// myCar contiene Vehicle@null
```

```
// myCar contiene Car@27c7f17c
```

Un oggetto Car eredita tutti i campi e i metodi degli oggetti Vehicle



ERRORI DI MEMORIA IN JAVA

Tuttavia Java non è del tutto memory-safe

```
Vehicle myCar1 = new Vehicle();
// myCar1 contiene Vehicle@37c7a17c

Vehicle myCar2;
// myCar2 contiene Vehicle@null
```

La chiamata a `myCar2.setFrameNumber(5)`
genera un errore `NullPointerException` !

Vehicle@37c7a17c

manufacturer Vehicle

setFrameNumber(int)

Vehicle@null

ERRORI DI MEMORIA IN JAVA (CONT.)



Il valore speciale `null` può essere usato dai programmatori

- ▶ Nelle condizioni: `myCar2 == null`, per verificare eccezioni da puntatore nullo
- ▶ Nelle assegnazioni: `myCar1 = null;`

// Perché?



DOMANDE E RISPOSTE

